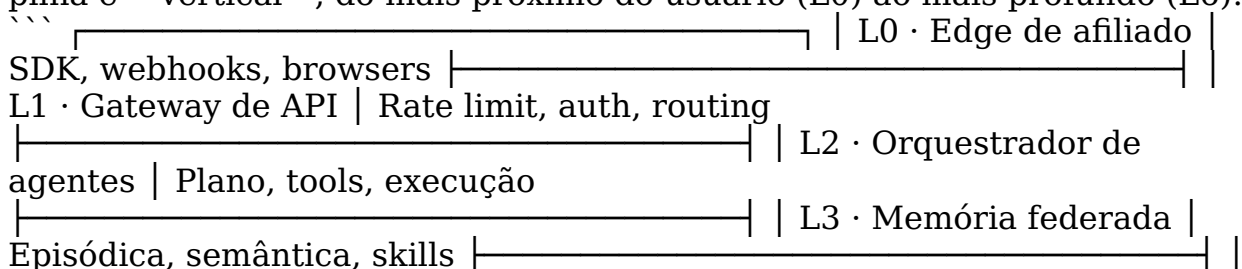
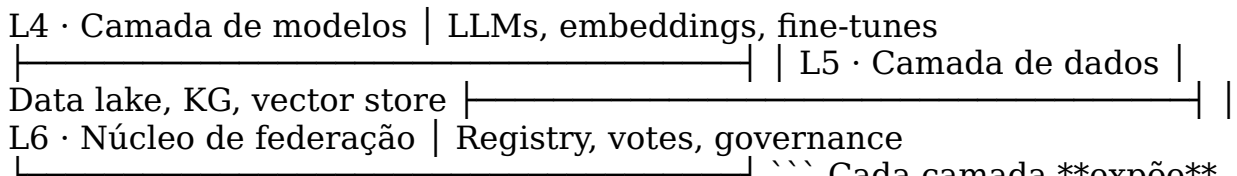


--- title: "IOAID — Arquitetura Profunda da Infraestrutura de IA Distribuída"
 subtitle: "Como Funciona, Camada por Camada, o Cérebro Operacional do Ecossistema Nexus" author: "MMN_IA Collective" version: "1.0.0" date: "2026-06-28" tags: [academia, ioaid, infraestrutura, arquitetura, distribuida]
 pattern: "MMN_IA" --- ![Capa — IOAID Arquitetura Profunda](../docs/ebooks/ACAD-apostila-12-ioaid-arquitetura-profunda.webp) **IOAID —**
Arquitetura Profunda da Infraestrutura de IA Distribuída ***Como Funciona,**
Camada por Camada, o Cérebro Operacional do Ecossistema Nexus* **Por**
MMN_IA Collective · Academ'IA **Nexus Affil'IA'te · 2026** **Sobre este**
documento **IOAID** é a sigla de **Infraestrutura de Orquestração Autônoma**
de Inteligência Distribuída — o nome técnico do que chamamos
 informalmente de "o motor da Nexus". Este documento é o mapa técnico
 profundo de cada uma das 7 camadas que compõem o IOAID: da borda
 (edge) onde os agentes atendem afiliados, até o núcleo (core) onde a
 federação é gerenciada. Se você é arquiteto de plataforma, operador de
 rede, ou afiliado que quer entender de verdade o que acontece "por trás do
 painel", este guia é seu ponto de entrada. **Sumário** > **1.** O que é o
 IOAID e por que ele é distribuído > **2.** As 7 camadas da pilha IOAID >
3. Camada L0 — Edge de afiliado (onde tudo começa) > **4.**
 Camada L1 — Gateway de API (a porta de entrada) > **5.** Camada L2 —
 Orquestrador de agentes (onde a mágica acontece) > **6.** Camada L3 —
 Memória federada (o cérebro de longo prazo) > **7.** Camada L4 —
 Camada de modelos (LLMs, embeddings, etc.) > **8.** Camada L5 —
 Camada de dados (data lake, knowledge graph) > **9.** Camada L6 —
 Núcleo de federação (o coração) > **10.** Como as camadas se conversam
 (request lifecycle completo) > **11.** Padrões de deployment: single-
 tenant vs multi-tenant > **12.** SLA e observabilidade por camada >
13. Como o IOAID escala horizontalmente > **14.** Failover e
 disaster recovery > **15.** Como customizar o IOAID para white-label >
16. Glossário de termos --- **1.** O que é o IOAID e por que ele é
 distribuído **IOAID** é a **infraestrutura computacional e lógica** que
 permite que o ecossistema Nexus funcione. Não é um monolito. É uma
federação de componentes que operam em camadas, cada uma com
 responsabilidade clara, contratos bem definidos, e SLAs próprios. **Por que**
distribuído? Em 2024, a Nexus operava com arquitetura monolítica: tudo
 num cluster Kubernetes único, em uma região, com failover limitado. Em
 2025, migramos para **arquitetura distribuída federada** por três razões: 1.
Resiliência: nenhuma região pode derrubar o ecossistema. 2.
Latência: edge nodes atendem afiliados com p99 < 200ms em qualquer
 continente. 3. **Soberania**: tenants em diferentes jurisdições têm dados
 em regiões apropriadas. O IOAID é o resultado dessa decisão. Cada camada
 pode escalar independentemente, falhar independentemente, e ser
 customizada independentemente. **2.** As 7 camadas da pilha IOAID **A**
 pilha é **vertical**, do mais próximo do usuário (L0) ao mais profundo (L6):





uma API para a camada acima e **consume** APIs das camadas abaixo. O

contrato é versionado e monitorado. **3. Camada L0 — Edge de afiliado** A

camada L0 é o que o afiliado **vê e toca**. Inclui: - **Web app** (`/`
`dashboard`) — interface principal. - **Mobile app** — iOS e Android. -

SDKs — Python, TypeScript, CLI. - **Webhooks** — notificações

outbound para sistemas do afiliado. - **API REST pública** — para

integrações customizadas. **Características-chave:** - Stateless: toda

requisição carrega auth + contexto. - Resiliente: opera offline por até 5min

com retry queue. - Multi-idioma: i18n com 11 idiomas em produção. -

Observabilidade: 100% das ações têm trace_id. **SLA:** disponibilidade

99.95%, p99 latência < 200ms para 90% das ações. **4. Camada L1 —**

Gateway de API A L1 é o **portão de entrada** de toda requisição. Faz: -

Autenticação — Bearer token, OAuth2, API key, mTLS. - **Rate**

limiting — por tenant, por IP, por global. - **Routing** — para o serviço

correto baseado em path/header. - **Logging estruturado** — toda

requisição é logada. - **Circuit breaker** — protege serviços downstream.

Stack típico: - Kong ou Tyk para API gateway. - Redis para rate limiting

state. - OpenTelemetry collector para tracing. - Prometheus para métricas.

SLA: disponibilidade 99.99%, latência adicionada < 10ms. **5. Camada**

L2 — Orquestrador de agentes A L2 é onde **agentes ganham vida**.

Inclui: - **Agent Registry** — onde cada agente é instanciado. - **Planner**

— decompõe objetivos em planos. - **Tool Router** — resolve qual tool usar

quando. - **Memory Manager** — coordena working/episodic/semantic. -

Executor — roda o plano de fato. **Características-chave:** - State

management distribuído (Redis + Postgres). - Fault tolerance: plano pode

retomar de onde parou. - Multi-tenancy: cada tenant tem namespace isolado.

- Auto-scaling: spawna workers sob demanda. **SLA:** disponibilidade

99.95%, execução de plano em <30s para 95% dos casos. **6. Camada L3 —**

Memória federada A L3 guarda **tudo que importa lembrar**: -

Memória episódica — eventos de conversa, decisões, traces. - **Memória**

estruturados e não estruturados** vivem: - **Data Lake** — S3 + Parquet para analytics de longo prazo. - **Knowledge Graph** — Neo4j para relações semânticas. - **Vector Store** — pgvector para busca por similaridade. - **Search Index** — Elasticsearch para full-text. **Casos de uso:** - Análise de cohort de afiliados. - Treinamento de modelos custom. - Auditoria de longo prazo. - Recomendações personalizadas. **SLA:** backups diários, retenção de 7 anos para audit logs. **9. Camada L6 — Núcleo de federação** A L6 é **onde a rede se governa**: - **Skill Registry** — onde skills são publicadas, votadas, removidas. - **Agent Reputation** — score de cada agente baseado em outcomes. - **Governance Engine** — implementa regras de plataforma. - **Federation Gateway** — interface com outras instâncias Nexus. **Princípio de design:** **nenhuma decisão crítica** é tomada por um único nó. Federation = consensus. **SLA:** disponibilidade 99.99%, decisão de governance <5s. **10. Como as camadas se conversam (request lifecycle completo)** Trace de uma requisição típica de afiliado: `` 1. Usuário clica "Disparar campanha" ↓ L0 (web app) 2. Web app chama API: POST /campaigns/{id}/dispatch ↓ L1 (gateway) 3. Gateway valida auth, rate limit, roteia ↓ L2 (orquestrador) 4. Orquestrador cria plano: [check budget, fetch audience, dispatch] ↓ L2 + L4 (modelos) 5. Planner usa LLM para validar plano ↓ L3 (memória) 6. Memória carrega contexto do tenant + histórico ↓ L2 (executor) 7. Executor roda cada step: - Check budget: query L5 - Fetch audience: query L3 - Dispatch: chama gateway do WhatsApp ↓ L1 8. Resposta volta ao usuário com trace_id `` **Tempo total:** ~800ms para uma campanha pequena. ~30s para uma campanha complexa com IA generativa. **11. Padrões de deployment: single-tenant vs multi-tenant** **Single-tenant:** - Um cliente Nexus por deployment. - Isolamento físico completo. - Mais caro, mais seguro. - Usado por: clientes enterprise. **Multi-tenant:** - Múltiplos clientes no mesmo deployment. - Isolamento lógico (namespace, RLS). - Mais barato, suficiente para 95% dos casos. - Usado por: 95% dos afiliados Nexus. **Híbrido (white-label):** - Multi-tenant mas com branding customizado. - Cada white-label vê apenas seus tenants. - Caso comum de parceria regional. **12. SLA e observabilidade por camada** | Camada | SLA | Observabilidade | |-----|-----|-----| | L0 Edge | 99.95% | RUM, error tracking, perf | | L1 Gateway | 99.99% | Request logs, latency | | L2 Orchestrator | 99.95% | Plan execution traces | | L3 Memory | 99.99% | Query latency, cache hit | | L4 Models | 99.9% | Token usage, cost | | L5 Data | 99.95% | Backup status, query plans | | L6 Federation | 99.99% | Vote latency, consensus | **SHO** monitora todas as camadas com 27 sensores (vide Apostila 11). **13. Como o IOAID escala horizontalmente** Cada camada escala independentemente: - **L0:** CDN + edge compute (Cloudflare, Vercel). - **L1:** API gateway stateless, escala com K8s HPA. - **L2:** workers stateless, estado em Redis/Postgres. - **L3:** sharding por tenant_id. - **L4:** multi-provider + cache agressivo. - **L5:** data warehouse escala com Snowflake/BigQuery. - **L6:** consensus distribuído com Raft. **Pattern 2026:** **scale-to-zero** para tenants pequenos, scale-to-millions para enterprise. **14. Failover e disaster recovery** Três níveis de failover: - **L1-L4:** hot standby em outra região, failover <30s. - **L5 (dados):** replica cross-region, RPO <5min, RTO <1h. - **L6 (federação):** quorum distribuído, tolera até 1/3 de nós offline. **DR drills** são trimestrais. Resultado esperado: **RTO <1h para SEV-4**. **15. Como customizar o IOAID para white-label** White-label pode customizar: - Branding visual (L0). - Domain e SSL (L0, L1). - Skills disponíveis (L3, L6). -

Modelos permitidos (L4). - Knowledge graph de domínio (L5). - Regras de governance (L6). ****Não pode**** customizar: SHO, audit logs, security baseline. ****16. Glossário de termos**** | Termo | Definição | |-----|-----| | ****IOAID**** | Infraestrutura de Orquestração Autônoma de IA Distribuída | | ****L0..L6**** | Camadas da pilha (0=edge, 6=federação) | | ****Edge**** | Computação próxima do usuário | | ****Gateway**** | Porta de entrada da API | | ****Orchestrator**** | Componente que coordena agentes | | ****Federation**** | Rede distribuída de nós interconectados | | ****Multi-tenant**** | Múltiplos clientes no mesmo deployment | | ****White-label**** | Plataforma com marca customizada | | ****RTO**** | Recovery Time Objective | | ****RPO**** | Recovery Point Objective | | ****Circuit Breaker**** | Padrão que isola serviço com falha | | ****HPA**** | Horizontal Pod Autoscaler (K8s) | | ****RUM**** | Real User Monitoring | | ****RLS**** | Row Level Security (Postgres) | | ****Quorum**** | Mínimo de nós para decisão em consensus | --- *© 2026 Nexus HUB57 · Academ'IA · Todos os direitos reservados* ****Versão 1.0.0 · Junho 2026 · Apostila Academ'IA #12 — IOAID — Arquitetura Profunda***